BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 04

> **Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

> Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:*Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

Type your answer here...

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Type your answer here...

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

Type your answer here...

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Type your answer here...

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

FACULTY OF COMPUTING

Q1. How does UCS differ from A* in terms of node prioritization?

Answer:

Uniform Cost Search (UCS) prioritizes nodes based only on the **actual cost from the start node**.

$$f(n) = g(n)$$

where g(n) is the cost from the start node to the current node.

A* Search considers both the actual cost and an estimated cost to the goal:

$$f(n) = g(n) + h(n)$$

where:

- g(n) = actual cost from the start to node n
- h(n) = heuristic estimate from node n to the goal
- f(n) = estimated total cost of the path

Therefore, UCS does not use information about how close a node is to the goal, while A* uses a heuristic to guide the search toward the goal.

Q2. Why is Manhattan distance admissible for 4-way movement? What happens if the heuristic overestimates?

Answer:

Manhattan distance is admissible for a grid where movement is restricted to **up, down, left, and right** because it never overestimates the minimum number of moves required to reach the goal.

The Manhattan distance is:

$$h(n) = | x_1 - x_2 | + | y_1 - y_2 |$$

For example, if the agent is at (0,0) and the goal is (3,4):

$$h(n) = | 0 - 3 | + | 0 - 4 | = 7$$

The agent needs at least 7 horizontal/vertical movements, assuming there are no obstacles.

If a heuristic **overestimates** the actual cost to the goal, it is no longer admissible. A* may then ignore a path that is actually optimal, meaning *A may fail to find the optimal/shortest path**.

Q3. If diagonal (8-way) movement is allowed, is Manhattan distance still admissible? What heuristic should you switch to?

Answer:

No. If diagonal movement is allowed, **Manhattan distance is no longer an appropriate admissible heuristic**, because diagonal movement can reach the goal using fewer moves than the Manhattan distance predicts.

For 8-way movement, a suitable heuristic is the **Euclidean distance**:

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

The Practical 04 implementation specifically asks us to implement both **Manhattan distance** and **Euclidean distance** as heuristic functions.

For example, from (0,0) to (3,4):

- Manhattan = 7
- Euclidean = 5.0

Q4. If there are multiple food items, how could you design a stronger heuristic than simple Manhattan distance to the nearest food?

Answer:

Instead of considering only the Manhattan distance to the nearest food, a stronger heuristic could consider the **overall remaining food locations** and the estimated cost of collecting them.

For example, the heuristic could consider:

- the distance to the nearest remaining food, and
- additional distances between remaining food items.

This would provide more information about the total remaining cost instead of considering only one food item.

A simple approach for multiple food items is to calculate the Manhattan distance from the current position to each remaining food item and use the **minimum distance** as the heuristic:

$$h(n) = \min_{f \in Food} Manhattan(n, f)$$

This gives an estimate based on the closest remaining food.